GROUP 18 · CRYPTOGRAPHY PROJECT

Task Division: Secure Cloud File Sharing Using Hybrid Cryptography

The work is split into four **equal, self-contained roles**. Each member presents slides, runs one part of the live demo, writes report sections, owns one part of the code, and prepares one area of theory from our syllabus for the viva. Each brief below is written so that the member can work on it independently. Read your own brief fully and skim the others, so that you can answer cross-questions.

At a glance (equal visible work)

	Saurav Singh	Rohan Dubey	Ujjawal	Aditya Raj
Role	Problem & system lead	Base-paper lead	Security analyst (attacks)	Solution lead
Slides presented	Title, Slide 1, Conclusion (Slide 6, bottom)	Slides 2 and 3	Slide 4	Slide 5, Results (Slide 6, top)
Speaking time (slides)	≈ 2.75 min	≈ 3 min	≈ 2 min	≈ 3.5 min
Live demo segment	D1 App tour, login & key vault, cloud view (≈ 2.5 min)	D2 Upload with both schemes, step by step (≈ 2.5 min)	D3 Attack Lab: attacks 1–3 (≈ 3 min)	D4 Share, revoke, results (≈ 2 min)
Total speaking	≈ 5.25 min	≈ 5.5 min	≈ 5 min	≈ 5.5 min
Report sections	Abstract, 1, 6, 8, App. A	2, 3, 9 (references)	4, 7.1–7.5	5, 7.6–7.8
Code owned	server/ (auth, files API, events), CloudView page	crypto-core/ base_aes_ecc.js, Dashboard (Files) page	server/routes/lab.js, AttackLab page, analysis.js, bench E1–E5	crypto-core/ proposed_ahe.js, fileops.js, Results page, bench E6–E9
Theory (course units)	Unit 2 Shannon entropy & secrecy; hash/PBKDF2 basics	Unit 1 number theory, Module 3 AES, Module 4 ECC	Module 3 block-cipher modes; cryptanalysis; avalanche	Unit 6 Diffie-Hellman & key mgmt; Module 5 hash & signatures; GCM

Running order on presentation day (≈ 21 min)

#	Part	Who	Time
1	Title slide: introduce group and topic	Saurav	0.5 min
2	Slide 1: introduction, threat model, problem statement	Saurav	1.5 min
3	Slide 2: technologies (AES, ECC, hybrid model of the paper)	Rohan	1.5 min
4	Slide 3: results reported by the paper + our reproduction	Rohan	1.5 min
5	Slide 4: three weaknesses (challenges) with evidence	Ujjawal	2 min
6	Slide 5: our improvement (AHE)	Aditya	2 min
7	Slide 6 (top): results after the fix	Aditya	1.5 min
8	Slide 6 (bottom): conclusion + future work, hand over to demo	Saurav	0.75 min
9	Demo D1 → D2 → D3 → D4	Saurav → Rohan → Ujjawal → Aditya	10 min

Demo preparation (everyone, once)

1. Install Node.js 20.19+ (or 22 LTS). In `app/` run `npm install`, then `npm test` (5 tests must pass).

2. Paste the MongoDB URL into `app/server/.env` (`MONGO_URI=...`). Run `npm start` and open http://localhost:5000; the console must show `[db] connected`. The demo users are created in that database on the first start.
3. Use two browser windows: a normal one for **alice** and an incognito/private one for **bob** (each window keeps its own login). Passwords are on the login page.
4. Before each segment, click **Clear** in the Crypto Inspector and choose **Step-by-step** mode. Click **Next** ▶️ to reveal one real step at a time while you explain it.
5. Rehearse twice with the timer. If the projector is small, zoom the browser to 80–90%.

Saurav Singh (S20230010219)

Role: Problem & system lead (introduction, threat model, server, conclusion)

Your task: Explain why cloud files need hybrid cryptography and what exactly we test, present the system architecture and the server side of the demo, and close the presentation.

1. Slides you present

Title slide (≈30 s)

- Introduce the group, the topic and the project type (research + application): we rebuilt a published scheme, attacked it, fixed it and measured it.

Slide 1: Introduction & problem statement (≈1.5 min)

- Cloud = our files on someone else's computer. Hybrid crypto: AES (fast) locks the file, ECC (public key) handles the key.
- Three attackers: outsider with a leaked copy, curious/malicious cloud insider, revoked user.
- Threat-model diagram: Alice → cloud → Bob/Charlie, attacker at the cloud.
- Problem statement + 4 objectives: rebuild, attack, fix, measure.

Slide 6, bottom: Conclusion (≈45 s)

- Three conclusion bullets + future work (ML-KEM, key transparency). Hand over to the demo.

2. Your live-demo segment

1. Show the architecture in one sentence: React client (all crypto in the browser), Express API, MongoDB + GridFS.
2. On the login page, set the Inspector to Step-by-step, log in as **alice**. Explain each step: the browser derives an auth token with PBKDF2 (the password is never sent); the server checks bcrypt; the browser opens the key vault with PBKDF2 (600,000 iterations) + AES-GCM.
3. Optional (30 s): open **Register**, type a password and show the Shannon-entropy meter (Unit 2), then register `demo1` and show the two P-256 key pairs generated in the browser.
4. Open **What the Cloud Sees**: explain that this is the raw database content (what a hacker with a leaked backup sees), and that the server log on this page updates live (Server-Sent Events).
5. Hand over to Rohan: “Now let's upload a file with both schemes.”

3. Report sections you write

- Abstract; Section 1 (Introduction: topic, motivation, importance, problem statement, objectives, threat model, course mapping).
- Section 6 (implementation of the proposed approach: code structure, application walk-through, experimental setup table).
- Section 8 (Conclusion) and Appendix A (how to run the demo).

4. Code you own (explain it line by line if asked)

- `app/server/src/` (`index.js`, `app.js`, `db.js`, `seed.js`, `models/*`, `routes/auth.js`, `routes/users.js`, `routes/files.js`, `lib/events.js`).
- `app/client/src/lib/session.jsx` (login/register flow), `pages/Login.jsx` (entropy meter), `pages/CloudView.jsx`.
- Key idea to explain: `deriveAuthToken` (PBKDF2 → sent to server, bcrypt stored) vs `sealPrivateKeys` (different PBKDF2 key with random salt, never leaves the browser).

5. Theory you must master (from our course)

- **Shannon entropy (Unit 2):** $H(X) = -\Sigma\, p(x) \log_2 p(x)$. For bytes the maximum is 8 bits/byte (all 256 values equally likely). Our ciphertext reaches 7.9997; the base ECB image ciphertext only 6.55.

- **Key equivocation:** H(K | C) is the attacker's remaining uncertainty about the key. In the base scheme H(K | public key) = 0, because the key is a function of public data.
- **Perfect secrecy:** P(M | C) = P(M); needs |K| ≥ |M| (one-time pad). Real systems, including ours, are computationally secure instead.
- **Password entropy:** charset estimate $L \cdot \log_2(N)$. Why PBKDF2 iterations slow down guessing (each guess costs 600,000 HMAC-SHA256 calls).
- **Hybrid cryptography idea:** symmetric for data + public key for key distribution (used in TLS, PGP).

6. Viva questions to prepare (with model answers)

Q. What is hybrid cryptography and why not use only RSA/ECC for the file?

Public-key operations are slow and work on small inputs only. A symmetric cipher (AES) encrypts the bulk data quickly, and public-key cryptography solves how to give the AES key to the right people.

Q. What is your threat model?

The cloud provider can read and modify everything it stores. An outsider may obtain a leaked copy plus all public keys. A revoked user must not read new versions. We trust the users' own browsers and TLS.

Q. What does an entropy of 7.9997 bits/byte mean?

The byte values of the ciphertext are almost perfectly uniform, so no statistical pattern is left. The maximum for bytes is 8. The base scheme's image ciphertext has 6.55 because ECB keeps repeated blocks.

Q. Why is H(K | P) = 0 in the base scheme?

The AES key is computed as a function of the public key P. Once P is known (it is public), the key is completely determined, so no uncertainty is left.

Q. Does the server ever see the password?

No. The browser sends PBKDF2(password, username-salt) as a login token; the server stores bcrypt(token). The private keys are sealed with a different PBKDF2 key (random salt, 600k iterations), so even the server's database cannot open them.

Q. Is your scheme perfectly secret?

No. Perfect secrecy requires a key at least as long as the message (one-time pad). AES-GCM and ECDH are computationally secure: breaking them needs infeasible work (about 2^{128} operations).

Q. What is GridFS?

MongoDB's way to store large binary files in 255 KB chunks. We store only ciphertext there.

7. Checklist

- ☐ Can present Title + Slide 1 + Conclusion without notes
- ☐ Can run D1 smoothly in Step-by-step mode
- ☐ Report sections Abstract/1/6/8/App A proof-read
- ☐ Can explain auth token vs vault key
- ☐ Can derive and explain the entropy formula

Rohan Dubey (S20230010207)

Role: Base-paper lead (technologies of the paper, its results, base implementation)

Your task: Explain the base paper's technologies (AES, ECC, the hybrid AES-ECC algorithm) and its reported results, show our faithful re-implementation, and run the upload demo with both schemes.

1. Slides you present

Slide 2: Technologies used in the paper (≈1.5 min)

- AES: symmetric, 128-bit blocks, 10 rounds of SubBytes, ShiftRows, MixColumns (GF(2^8)), AddRoundKey.
- ECC: curve $y^2 = x^3 + ax + b$, private n, public P = n·G, ECDLP is hard; 256-bit ECC ≈ 3072-bit RSA.
- The paper's 4 steps (Box 1 + Box 2) and its claims; the authors' repository is empty, so we implemented it ourselves.

Slide 3: Results of the work (≈1.5 min)

- The paper's Table 7 chart (hybrid 2.40–2.60 s vs AES 3.42–3.60 s), plus its claims (avalanche 61/64%, ECC:RSA 1:12, integrity).
- Our reproduction works: 100/100 files, ≈928 MB/s, P-256 keygen 0.28 ms vs RSA-3072 145 ms.
- End with: “It is fast. But is it secure?” and hand over to Ujjawal.

2. Your live-demo segment

1. Logged in as alice, Inspector in Step-by-step mode. Choose `payment_order.txt`, scheme **Base**, share with bob, click **Encrypt & upload**.
2. Reveal the steps: the browser sends the PLAINTEXT; the CSP takes the x-coordinate of alice's public key, hashes it to get the AES key (point out: “no secret input”); encrypts with AES-ECB; 0 tag stored.
3. Clear, select **Proposed**, upload the same file. Reveal: random DEK, AES-GCM with nonce and tag, ECDH key-wrap for alice and bob (show the shared secret), ECDSA signature, “DEK wiped”, and the server reporting it cannot decrypt.
4. Open the proposed file as alice: show the 4 verification steps (signature, hash, unwrap with ECDH, GCM tag).
5. Hand over to Ujjawal: “Both files are in the cloud. Let's attack them.”

3. Report sections you write

- Section 2 (overview of the article: technologies, methodology, reported results, advantages).
- Section 3 (implementation of the existing approach: tools, decisions table, code excerpt, setup, results).
- Section 9 (references): check every reference is complete and consistently formatted.

4. Code you own (explain it line by line if asked)

- `app/crypto-core/src/base_aes_ecc.js`: `baseKeygen` (Box 1: P = n·G), `deriveAesKeyFromPublic` (Box 2), `baseEncrypt/baseDecrypt` (AES-ECB), traced versions.
- `app/client/src/pages/Dashboard.jsx` (Files page) and `uploadBase` in `lib/fileops.js`.
- Be ready to justify every decision in Table 4 of the report (curve, KDF, mode, server-side encryption) with a quote from the paper.

5. Theory you must master (from our course)

- **Number theory (Unit 1):** modular arithmetic, modular inverse via the extended Euclidean algorithm, used in ECC point addition (slope $\lambda = (y_2 - y_1)/(x_2 - x_1) \bmod p$).
- **AES (Module 3):** state 4×4 bytes; S-box = inverse in GF(2^8) + affine map; MixColumns = matrix over GF(2^8) with $m(x) = x^8+x^4+x^3+x+1$; key expansion (44 words for AES-128); 10/12/14 rounds.

- **ECC (Module 4):** point addition and doubling, scalar multiplication by double-and-add, ECDLP; NIST P-256; why ECC keys are smaller than RSA keys (best attack on ECC is Pollard's rho ≈ √n, while RSA has sub-exponential factoring algorithms).
- **RSA basics for comparison:** n = pq, e·d ≡ 1 mod φ(n). RSA-3072 key generation is slow because it needs large primes.

6. Viva questions to prepare (with model answers)

Q. Explain one AES round.

SubBytes replaces each byte through the S-box (non-linear, confusion), ShiftRows rotates the rows, MixColumns mixes each column with a fixed matrix over GF(2^8) (diffusion), and AddRoundKey XORs the round key. AES-128 has 10 rounds, and the last round skips MixColumns.

Q. What is P = n·G and why is it secure?

G is a fixed base point and n is the private number. n·G means adding G to itself n times (done quickly with double-and-add). Getting n back from P is the elliptic-curve discrete-log problem, for which no efficient algorithm is known.

Q. Why is 256-bit ECC as strong as 3072-bit RSA?

The best known attack on ECC (Pollard's rho) takes about √n ≈ 2^{128} steps for a 256-bit curve. RSA can be attacked with sub-exponential factoring (the number field sieve), so it needs a 3072-bit modulus for 128-bit security (NIST SP 800-57).

Q. How exactly did you build the AES key in the base scheme? Isn't that your mistake?

The paper's Box 2 says AES encryption is performed “using the public key which is generated by ECC”, and it says the public key is known by everyone. We used SHA-256 of P's x-coordinate. Any function of the public key has the same flaw, because the attacker can compute it too.

Q. Why is a “64-bit AES key” wrong?

FIPS-197 defines AES only for 128, 192 and 256-bit keys. A 64-bit AES does not exist, so the paper's 64-bit rows cannot be real AES.

Q. Can a hybrid AES+ECC scheme be faster than AES alone?

Not if it uses the same AES implementation, because it does all the AES work plus ECC work. A faster result means different implementations or data were compared.

Q. Why did you write your own implementation?

The repository linked in the paper contains only a README, with no code. We followed Box 1 and Box 2 and documented every choice the paper leaves open.

7. Checklist

- ☐ Can present Slides 2–3 without notes
- ☐ Can run D2 in Step-by-step mode
- ☐ Sections 2, 3 and 9 proof-read
- ☐ Can draw one AES round and the ECC point-addition picture on the board
- ☐ Can justify the base-scheme implementation choices

Ujjawal (S20230010245)

Role: Security analyst (weaknesses, attacks, statistical analysis)

Your task: Find and prove the weaknesses of the base paper. Present the three challenges, run the live attacks, and explain the measurements (entropy, repeated blocks, avalanche).

1. Slides you present

Slide 4: Weaknesses of the paper, 3 challenges (≈2 min)

- C1: the key comes from the PUBLIC key, so 100/100 files were decrypted with public data only (Kerckhoffs; H(K|P)=0).
- C2: no integrity check and ECB-style mode: 0/1,250 tampering attempts detected; image patterns stay visible; payment amounts swapped.
- C3: no real sharing or revocation: the cloud encrypts, so it sees the plaintext; revocation changes no key; problems in the paper's evaluation (64-bit AES, avalanche >50%, power = time, empty repository).

2. Your live-demo segment

1. Open **Attack Lab**; select the base and proposed `payment_order.txt`. Inspector: Step-by-step.
2. Attack 1: click **Attack the BASE file** and reveal the attacker steps (gets the ciphertext → reads alice's public key → applies the paper's rule → full plaintext). Then **Attack the PROPOSED file**: all candidate keys fail the GCM tag.
3. Attack 2: choose "Swap ciphertext blocks 2 ↔ 4". **Tamper BASE** → **Open as me**: the amounts are swapped and there is no warning. **Tamper PROPOSED** → **Open as me**: BLOCKED, and the Inspector shows that the hash no longer matches the signed manifest. Click **Restore originals**.
4. Attack 3: **Run with sample logo**: the logo stays visible in the AES-ECB ciphertext and the GCM ciphertext looks like noise. Read out the entropy and repeated-block numbers under the images.
5. Hand over to Aditya: "Now the sharing and revocation part."

3. Report sections you write

- Section 4 (challenges and limitations: C1, C2, C3, evaluation issues, summary table).
- Sections 7.1–7.5 (summary table, attack 1, attack 2, entropy/patterns/determinism, avalanche re-evaluation).

4. Code you own (explain it line by line if asked)

- `app/server/src/routes/lab.js` (public-key attack, tamper/restore, ECB endpoint).
- `app/client/src/pages/AttackLab.jsx` and `app/crypto-core/src/analysis.js` (Shannon entropy, repeated-block ratio, chi-square, bit difference).
- `app/bench/run.js` experiments E1–E5 (how "detected" vs "silent corruption" is counted).

5. Theory you must master (from our course)

- **Block cipher modes (Module 3):** ECB, CBC, CFB, OFB, CTR. ECB encrypts equal blocks to equal ciphertext, so patterns leak. CBC/CTR need an IV/nonce. No basic mode gives integrity on its own.
- **Malleability:** in ECB, ciphertext blocks can be swapped or deleted; in CBC, flipping a bit in C_i flips the same bit in P_{i+1}; in CTR, flipping a ciphertext bit flips the same plaintext bit.
- **Kerckhoffs' principle** and classical-cipher lessons (frequency analysis works because structure survives encryption, just like ECB).
- **Avalanche / strict avalanche criterion:** a 1-bit change should flip each output bit with probability ½, so 50% is ideal. Our AES measurement is ≈49.9%.
- **Statistics:** Shannon entropy per byte, repeated-block ratio, chi-square against the uniform distribution (≈255 expected for random bytes).

6. Viva questions to prepare (with model answers)

Q. Why does ECB leak the picture?

Each 16-byte block is encrypted separately with the same key, so equal pixel blocks (flat colours) produce equal ciphertext blocks. The outline of the image therefore survives encryption.

Q. How could the cloud swap the payment amounts without knowing the key?

In ECB each block decrypts independently. Swapping ciphertext blocks 2 and 4 swaps plaintext blocks 2 and 4, which hold the two amount lines. With no MAC, nothing detects the change.

Q. Would switching the base scheme to CBC fix it?

No. The key would still be computed from the public key (C1). CBC is also malleable and, without a MAC, open to padding-oracle attacks (Vaudenay 2002). Only authenticated encryption fixes integrity.

Q. The paper reports 64% avalanche. Isn't higher better?

No. The ideal is 50%: each output bit should flip with probability ½. 64% means the bits are biased (a 100% flip would simply be the complement). Our measurement of AES gives ≈49.9%.

Q. What is Kerckhoffs' principle and how does the base scheme break it?

A cryptosystem should stay secure even if everything except the key is public. The base scheme's key is itself computed from public information, so nothing secret remains.

Q. How did you decide that a tampering attempt was “detected”?

Detected means decryption raised an error and released nothing. Silent corruption means it returned plaintext different from the original. The base scheme detected 0; ours detected 100%.

Q. Why is the base ciphertext entropy 6.55 and not 8 even though AES is used?

AES-ECB maps the many repeated plaintext blocks to the same few ciphertext blocks, so some byte patterns are much more frequent than others. A good mode with a fresh nonce makes every block different.

7. Checklist

- ☐ Can present Slide 4 without notes
- ☐ Can run D3 (attacks 1–3) and restore the originals
- ☐ Sections 4 and 7.1–7.5 proof-read
- ☐ Can explain ECB/CBC/CTR on the board
- ☐ Can explain the avalanche criterion and the entropy numbers

Aditya Raj (S20230030364)

Role: Solution lead (proposed AHE scheme, improved results)

Your task: Design and explain the improved scheme (random key, AES-GCM, ECDH key wrapping, ECDSA signatures, re-keying), present the improved results, and demonstrate sharing and revocation.

1. Slides you present

Slide 5: Our fix, the authenticated hybrid envelope (≈2 min)

- 5 steps: random DEK → AES-256-GCM (+tag) → lock the key for each user with ECDH + HKDF → owner signs with ECDSA → the cloud stores only ciphertext. Each step names the challenge it fixes.
- Opening: verify signature → check hash → unlock with my private key → decrypt + tag check. Any failure → BLOCKED.
- Sharing adds one locked key (2.7 ms). Revocation re-keys to version 2 (7.6 ms/MiB). Everything comes from course topics.

Slide 6, top: Results after the fix (≈1.5 min)

- Table: 100/100 → 0/100; 0% → 100% tampering detected; 98% → 0% repeated blocks; entropy 6.55 → 8.00; revocation impossible → possible.
- Cost: ≈3 ms per file; 16 MiB in 41 ms (≈390 MB/s). Hand over to Saurav for the conclusion.

2. Your live-demo segment

1. As alice: upload `salary_records.csv` with **Proposed**, shared with bob and charlie.
2. In bob's window (incognito): **Refresh**, **Open** the file. Reveal the steps: ECDSA verify, hash check, ECDH unwrap (shared secret S = d·R), GCM decrypt.
3. As alice: **Share / Revoke** → **Revoke** bob. Reveal: decrypt, new random DEK, re-encrypt as version 2, wrap only for alice + charlie, sign.
4. In bob's window: **Refresh**. The file shows “access revoked”. Click **Open**: the server still sends the new ciphertext, but bob has no wrapped key, so access is DENIED by cryptography.
5. Open **Results**: tampering chart, entropy chart, time vs size, avalanche table. End: “All numbers are generated by `npm run bench` with the same code.”

3. Report sections you write

- Section 5 (proposed approach: selected challenges, design principles, building blocks, pseudocode, security analysis, limitations).
- Sections 7.6–7.8 (performance, sharing and revocation, challenge-by-challenge outcome).

4. Code you own (explain it line by line if asked)

- `app/crypto-core/src/proposed_ahe.js`: `encryptFile`, `decryptFile`, `wrapDek` / `unwrapDek` (ECDH + HKDF + AES-GCM), `addRecipient`, `rekeyFile`, `publicKeyAttackOnProposed`.
- `app/client/src/lib/fileops.js` (`uploadProposed`, `openFile`, `shareFile`, `revokeUser`), `app/client/src/pages/Results.jsx`.
- `app/bench/run.js` experiments E6–E9 (performance, sharing, revocation, primitives) and `app/crypto-core/test/crypto.test.js`.

5. Theory you must master (from our course)

- **Diffie-Hellman (Unit 6) → ECDH:** Alice sends R = r·G, Bob has Q = d·G; both compute S = r·Q = d·R = rd·G. An eavesdropper sees R and Q but cannot get S (ECDH problem).
- **Key management (Unit 6):** hybrid KEM/DEM (ECIES), one wrapped key per recipient, revocation by re-keying, why the server must not hold keys.

- **Hash functions (Module 5):** SHA-256 properties (pre-image and collision resistance, avalanche); HKDF = HMAC-based extract-then-expand.
- **Digital signatures (Module 5):** DSA/ElGamal-style signatures; ECDSA = DSA on elliptic curves ($r = (k \cdot G).x \bmod n$, $s = k^{-1}(H(m) + d \cdot r) \bmod n$); the nonce k must never repeat.
- **AES-GCM:** CTR mode for encryption + GHASH (polynomial evaluation in $GF(2^{128})$) for a 128-bit tag; the 96-bit nonce must never repeat under the same key; associated data is authenticated but not encrypted.

6. Viva questions to prepare (with model answers)

Q. How does GCM detect tampering?

It computes a 128-bit tag over the ciphertext and the associated data using GHASH, keyed from the AES key. Decryption recomputes the tag; if even one bit changed the tags differ and decryption is refused. A forgery succeeds with probability about 2^{-128} per try.

Q. Explain your key wrapping. Where is Diffie-Hellman?

For each recipient we pick a fresh r, publish $R = r \cdot G$, compute $S = r \cdot Q$ with the recipient's public key Q, derive a KEK with HKDF, and encrypt the DEK with AES-GCM. The recipient computes $S = d \cdot R$ with their private key d. This is Diffie-Hellman key agreement on the elliptic curve, as in ECIES.

Q. Why HKDF? Why not use S directly as the key?

S is a curve coordinate, not a uniformly random string. HKDF extracts its entropy and binds the key to the context (fileId, version, recipient, R), so keys for different files or versions are independent.

Q. GCM already authenticates. Why sign as well?

Every recipient knows the DEK, so any recipient could produce a valid GCM ciphertext. Only the owner holds the ECDSA private key, so the signature proves who uploaded the file and protects the manifest (recipients, version, hash).

Q. How does revocation work, and what are its limits?

We decrypt, generate a new DEK, re-encrypt as version+1 and wrap the key only for the remaining users. The old wrapped key cannot open the new version (the version is bound in the associated data). A revoked user can still keep data they already downloaded.

Q. What does security cost?

About 3 ms per file for two ECDH operations, one signature and key imports, plus GCM and SHA-256 over the data: 16 MiB takes 41 ms instead of 17 ms. That is still hundreds of MB/s, faster than any upload.

Q. Is your scheme novel?

No. It is a careful combination of standard constructions (ECIES-style KEM, AEAD, signatures), similar to what modern secure messaging and file-encryption tools use. Our contribution is finding the flaws in the published scheme, applying these standard fixes, implementing them, and measuring both schemes under identical attacks.

7. Checklist

- ☐ Can present Slide 5 and the results of Slide 6 without notes
- ☐ Can run D4 with two browser windows
- ☐ Sections 5 and 7.6–7.8 proof-read
- ☐ Can write the ECDH and ECDSA equations on the board
- ☐ Can explain why GCM + signature are both needed

Questions anyone may be asked

Q. Which paper did you choose and why?

Rehman et al., “Hybrid AES-ECC Model for the Security of Data over Cloud Storage”, Electronics 10(21):2673, 2021 (MDPI, peer reviewed, open access). It matches our topic, uses AES and ECC from our syllabus, and its full text is available.

Q. Is this a survey?

No. It is a research + application project: we implemented one published scheme, analysed it with experiments, and implemented and evaluated an improvement.

Q. Why MERN?

It gives a realistic cloud setting: a browser client (where end-to-end encryption must happen), a REST server, and a document database with GridFS for blobs. WebCrypto in the browser and Node's OpenSSL give standard, audited primitives.

Q. Did you use any cryptography library you don't understand?

We used only standard primitives (AES, SHA-256, ECDH, ECDSA, HKDF, PBKDF2) from the browser's Web Crypto API and Node's OpenSSL. We did not invent any cipher; the design and the analysis are ours.

Q. Where do your numbers come from?

From `app/bench/run.js`, which runs the same crypto code as the app. Results are saved in `results.json`, and the slides, report and app read that file, so all numbers match.